



2026
Software
Architecture
Specifications
Uni Textbook Marketplace

For Demo 4
Presented by:



Software Architecture Specifications (SAS)

Project: Uni Textbook Marketplace
Team: NexusDev
Client: Agile Bridge
University: University of Pretoria - COS 301 Software Engineering
2026
Document version: 3.0 - Demo 4
Last updated: 29 September 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Contents

1	Introduction	5
2	Architectural Requirements	6
2.1	Architectural Patterns	6
2.2	Design Patterns	8
2.3	Constraints	8
2.4	Architectural Diagram	9
2.5	Mapping Quality Requirements to Architectural Decisions	10
3	Technology Requirements	12
4	API Contracts	15
4.1	Carried Over from Demo 1 (confirmed still in use)	15
4.2	Auth API Contracts	15
4.2.1	POST /auth/register	15
4.2.2	POST /auth/verify-otp	15
4.2.3	POST /auth/login	16
4.2.4	POST /auth/set-password	16
4.2.5	GET /auth/me	17
4.2.6	POST /auth/logout	17
4.2.7	GET /auth/universities	17
4.2.8	POST /auth/refresh	17
4.3	Listings API Contracts	17
4.3.1	POST /listings	17
4.3.2	GET /listings	18
4.3.3	GET /listings/mine	18
4.3.4	GET /listings/:id	18
4.4	Admin API Contracts	18
4.4.1	GET /listings/admin/all	18
4.4.2	PATCH /admin/listings/:id/approve	19
4.4.3	PATCH /admin/listings/:id/reject	19
4.5	Modules API Contract	19
4.5.1	POST /modules	19
4.5.2	GET /modules/faculties	19
4.6	New for Demo 2	19
4.7	Listing API Contract	19
4.7.1	PATCH /listings/editlist	19
4.8	Module API Contract	20

4.8.1	GET /modules (extended filters)	20
4.9	Saved Searches API Contract	20
4.9.1	POST /saved-searches/	20
4.9.2	GET /saved-searches/me	21
4.9.3	GET /saved-searches/id	21
4.9.4	DELETE /saved-searches/id	21
4.9.5	GET /saved-searches/me/email	21
4.9.6	GET /saved-searches/me/role	22
4.10	Wishlist API Contract	22
4.10.1	POST /wishlist/:listingId	22
4.10.2	DELETE /wishlist/:listingId	22
4.10.3	GET /wishlist/mine	22
4.11	Admin API Contract	22
4.11.1	GET /admin/audit-log	23
4.11.2	GET /admin/emails	23
4.12	Image API Contract	23
4.12.1	POST /images/upload	23
4.12.2	GET /images/list	23
4.12.3	DELETE /images/delete	23
4.13	Conversations API Contract	24
4.13.1	POST /conversations	24
4.13.2	POST /conversations/mine	24
4.13.3	POST /conversations/:id/messages	24
4.13.4	GET /conversations/:id/messages	24
4.14	Demo 3 API Contracts:	25
4.14.1	POST /reports	25
4.14.2	GET /admin/reports	25
4.15	Admin Moderation API Contract	25
4.15.1	PATCH /admin/reports//dismiss	25
4.15.2	PATCH /admin//ban	25
4.16	Notifications API Contract	26
4.16.1	GET /notifications/mine	26
4.16.2	PATCH /notifications/:id/read	26
4.16.3	PATCH /notifications/read-all	26
4.16.4	DELETE /notifications/:id/delete	26
4.17	Listing Status API Contract	26
4.17.1	PATCH /listings/:id/status	27
4.18	Demo 4 API Contracts	27
4.18.1	POST /vision/extract-text	27

5	Non-Functional Requirements (NFR) Testing	30
5.1	Testing Overview	30
5.2	NFR Traceability Matrix	30
5.3	Test Results Summary	32
5.4	Load Test Results	32
5.5	Accessibility Test Results	33
5.6	Testing Tools Used	33
5.7	Known Issues and Action Plan	34
5.7.1	Critical Issues (Need Immediate Fix)	34
5.7.2	Untested NFRs (Need Testing)	34
6	Deployment	35
6.1	Overview	35
6.2	Live URLs	36
6.3	Deployment Diagram	36
6.4	CI/CD Pipeline Diagram	37
6.5	Deployment Requirements Status	38
6.6	Custom Domain and DNS	38
6.7	Rollback Strategy	39
6.8	Issues Encountered and the Decisions That Came Out of Them	39

1. Introduction

The Uni Textbook Marketplace is a web app that lets verified university students buy, sell, and swap second-hand textbooks in a way that's actually organised around modules, instead of just being a generic classifieds board. The SRS explains what the system needs to do. This document is about how we actually built it to do that: the architectural and design patterns we went with, the tech stack, our API contracts, and how everything gets deployed.

Roughly speaking, the backend is a NestJS modular monolith, the frontend is Next.js, and Postgres (hosted through Azure) handles the data. Messaging is kept separate and runs through Firebase Firestore instead. Redis, running as its own container, backs the job queue that opens and closes real-time auctions on time. Azure AI Vision does the text recognition behind AI-assisted listing creation. Everything runs on Azure Container Apps, split into a production and a staging environment, and GitHub Actions handles both the quality checks and the actual deployment. The frontend is also reachable on a custom domain, <https://www.unitextbookmarketplace.co.za>.

2. Architectural Requirements

2.1 Architectural Patterns

Modular Monolith (Core Backend)

The backend is a NestJS modular monolith. Rather than splitting everything into separate microservices right from the start, all the core domain logic (Auth, Listings, Moderation, Reports and Bans, Modules, Saved Search, Wishlist, Notifications, Audit Log, Auction, Vision, Bundle) lives in one NestJS app, just split into bounded modules. Each module has its own controller, service, and module file, and modules aren't allowed to inject each other's services directly. We went with this mainly because we're a small team and the domain is pretty well understood at this stage, so going full microservices this early would just be extra operational overhead for not much gain. Since the modules are already cleanly separated, if we ever did need to split into microservices later, it wouldn't mean a big rewrite.

External Microservice (Messaging)

In-app messaging (UC4) runs through Firebase Firestore as its own separate microservice, kept outside the core monolith on purpose. The frontend talks to Firestore directly through the client SDK, so the NestJS backend isn't sitting in the middle of that traffic at all. That means if Firestore ever goes down, it doesn't take the rest of the platform with it. That's a graceful degradation boundary we built in deliberately, not something that just happened. The auction feature reuses the same boundary for its live bid updates: the backend writes the current highest bid to Firestore only after the Postgres transaction has committed, and the frontend listens to that document directly.

Stateless API Layer

The NestJS API doesn't keep any session state on the server side. Auth works through a JWT plus an HTTP-only cookie, and the cookie settings change depending on the environment (`sameSite: 'none'` and `secure: true` once it's deployed, since frontend and backend sit on different hostnames there, and `sameSite: 'lax'` locally where that isn't an issue). Because there's no server-side session state, any backend instance can handle any request, which is what actually makes horizontal scaling possible without needing sticky sessions.

Read-Heavy Performance Strategy

People browse and search listings way more than they create new ones, so the schema has indexes on the columns that get used most in filters and sorting: a composite index on (`module_id`, `price`), plus separate indexes on `condition`, `annotation_level`, `status`, `seller_id`, `reviewed_by`, and `created_at`. Books get indexed on `isbn`, (`author`, `title`), and `edition`. Audit log entries are indexed on (`entity_type`, `entity_id`) and

performed_at.

Repository Pattern (Data Access)

Services never write raw SQL directly. All database access goes through TypeORM repositories, so the actual Postgres queries stay abstracted away from the business logic. This is basically the object-persistence pattern we covered in the architecture lectures (L11), where a DB manager layer sits between the business objects and the actual database, so the business logic doesn't need to know or care what's underneath it.

Event-Driven Job Scheduling (Auctions)

Auctions have to open and close at an exact moment without a human doing anything. Instead of a cron job scanning the auction table every few seconds, each auction schedules delayed jobs on a Redis-backed queue (BullMQ): an activation job when the start time is in the future, and a close job for every auction. A job fires once, exactly when it is needed, so there is no wasted polling. Because a bid in the last 30 seconds extends the end time (anti-sniping), the close job is self-correcting. When it fires it re-reads the auction, and if the end time has moved it simply reschedules itself for the new time instead of closing. This is simpler and safer than cancelling and re-adding jobs, which would create a race between two extensions. Redis runs as a plain container inside the Container Apps Environment with internal ingress only, so it is not reachable from the internet.

Pessimistic Locking (Bid Placement)

Two students can bid at the same instant, so placing a bid runs inside one Postgres transaction that locks the auction row with `SELECT ... FOR UPDATE`. Concurrent bids are serialised, the second one is validated against the first one's result, and exactly one of two identical bids is accepted. The locked query deliberately has no joins, because Postgres cannot lock the nullable side of an outer join, and no external calls are made while the lock is held.

Graceful Degradation (AI-Assisted Listing Creation)

AI-assisted listing creation has two independent pipelines, and both are optional. The OCR pipeline is backend-mediated: the frontend sends the uploaded photo's URL to the Vision module, which calls Azure AI Vision, fuzzy-matches the text against the `Book` table and returns a suggestion. The crop pipeline runs entirely in the browser, using a dependency-free flood fill to remove uniform backgrounds and detect four corners, which the seller can drag before confirming. If the OCR call fails or times out, or the background is too busy to detect, the seller gets a short non-blocking message and manual entry works exactly as it always did.

2.2 Design Patterns

- **Decorator Pattern.** NestJS guards (`@UseGuards(JwtAuthGuard, RolesGuard)`) add auth checks onto a controller method without touching what the method actually does. Keeps the security stuff separate from the endpoint's actual job.
- **Builder Pattern.** TypeORM's `createQueryBuilder()` is used for listing search. We start with a base query and keep adding `andWhere()` clauses depending on which filters were actually sent in the request, instead of writing a separate hardcoded query for every possible combination of filters.
- **Factory Pattern.** `ListingResponseFactory` is a small factory that turns `Listing` entities into the objects the API actually returns, adding stuff like a formatted price or a status label that isn't stored in the DB itself. Keeps the response shape the same across every listing endpoint.
- **Producer-Consumer (Job Queue).** The Auction service produces delayed `activate-auction` and `close-auction` jobs, and a separate processor consumes them. Scheduling and execution are decoupled, and the processor can be tested on its own.
- **Observer.** Firestore listeners push new messages and new highest bids to connected clients, so the UI reacts to changes instead of polling for them.
- **Fuzzy Matching Utility.** `book-matcher.util.ts` scores OCR text against candidate books and only returns a match above a confidence threshold, keeping the messy matching logic out of the controller and the service.

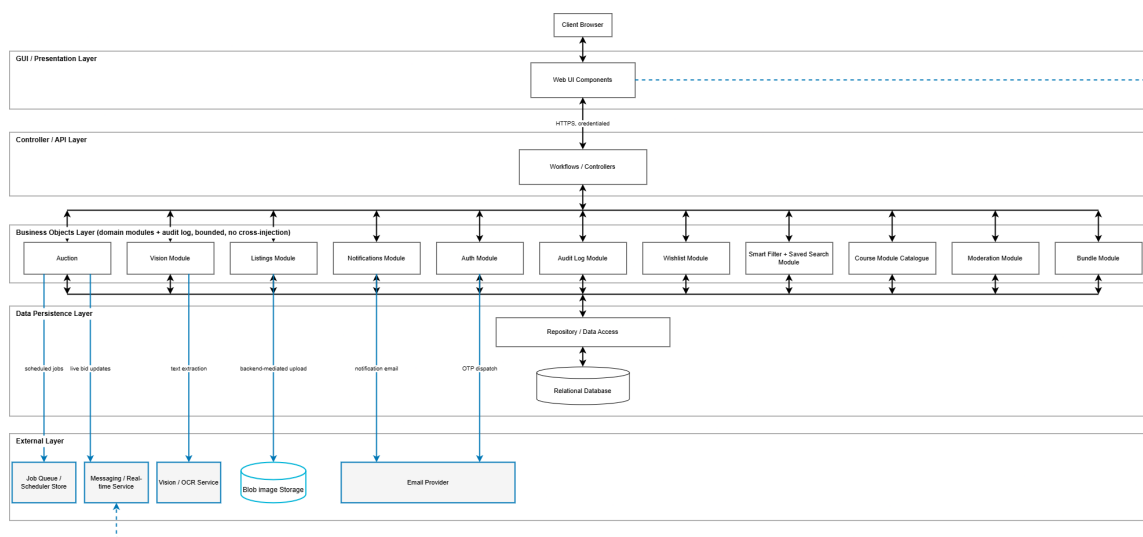
2.3 Constraints

Constraint	Detail
Hosting	Azure, under Agile Bridge's subscription (through our client contact Kyle). We manage the actual deployment but not the subscription/tenant itself.
Authentication	Custom OTP email verification, not a managed identity provider like Azure B2C.
Email allowlist	Student email domains of fifteen South African universities, with <code>@tuks.co.za</code> (University of Pretoria) as the primary domain in active use. The list is configurable through backend environment variables.
No payment processing	The platform only sets up meetup-based exchanges. No payment gateway, escrow, or any kind of financial transaction.

Constraint	Detail
ISBN metadata	No external ISBN lookup API. Sellers can type book details in, or photograph the cover and let OCR pre-fill them from books already in our own database.
AI service quota	Azure AI Vision runs on the free (F0) tier, which has low per-minute and monthly call limits. OCR is only called when a seller chooses to scan a cover, and automated tests use fixtures instead of the live service.
Redis availability	A single Redis container with no replication or failover, which is acceptable for a time-boxed project. If it is down, auctions cannot schedule or close, but the rest of the platform keeps working.
No mobile app	Web only for now. A React Native app is more of a stretch goal, not something we're actually building.
Multi-university support	The allowlist covers fifteen universities, but <code>tuks.co.za</code> is the only one in active use. The data model itself supports more than one university.

2.4 Architectural Diagram

This is a tech-neutral view of the main components in the system and how they talk to each other.



Notice how messaging doesn't go through the NestJS gateway at all, that's the graceful degradation boundary from §2.1, shown visually here. The diagram stays technology-neutral, so Redis appears as the job queue and scheduler store, and Azure AI Vision appears as the OCR service. Both sit in the external layer, reached only by the Auction and Vision

modules respectively.

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
Fast multi-filter listing search	Composite and single-column Postgres indexes on the columns used for listing search and sorting. Building the query with the builder pattern avoids unnecessary full-table scans.
Core features stay up if messaging fails	Messaging is kept separate as an external Firestore microservice, hit directly by the frontend, so if Firestore goes down it can't cascade into Auth, Listings, or Moderation.
Horizontal scalability	Stateless JWT-based auth means any backend instance can serve any request. Azure Container Apps can just add replicas without needing sticky sessions.
Only verified students can create listings, only admins can moderate	OTP verification happens at registration. <code>JwtAuthGuard</code> and <code>RolesGuard</code> (Decorator pattern) sit on every protected route.
Modular, low-coupling codebase	NestJS bounded modules (Auth, Listings, Moderation, Reports and Bans, Modules, Saved Search, Wishlist, Notifications, Audit Log, Auction, Vision, Bundle), with no module directly injecting another module's service.
WCAG AA accessibility	Checked through automated Lighthouse audits. Currently around 95% AA compliant.
Measurable backend test coverage	<code>ci.yml</code> runs <code>npm run test:cov</code> against a real, temporary Postgres 15 container on every push and PR to <code>main/develop</code> . Coverage gets uploaded to Codecov.
Code quality and static analysis	SonarCloud runs on every push to <code>main/develop</code> and on every PR targeting either branch.
Reproducible, environment-parity deployment	Two fully separate Container App pairs for production and staging, each with its own database, both deployed automatically through SHA-tagged images (see §6).

Quality Requirement	Architectural Decision
Only one winning bid under simultaneous bidding	Bid placement runs in one Postgres transaction that locks the auction row (pessimistic locking), so concurrent bids are serialised and exactly one of two identical bids is accepted.
Auctions open and close on time without polling	Delayed jobs on a Redis-backed queue fire once at the exact time. The close job re-checks the end time when it fires, so an anti-sniping extension can never cause an early or double close.
AI-assisted listing creation can never block selling	OCR is an optional, time-limited, backend-mediated pre-fill, and the crop runs in the browser. If either fails, manual entry works exactly as before.
Cloud credentials never reach the client	The Vision endpoint and key, the storage credentials and the database connection string live only in the backend as Container Apps secrets.

3. Technology Requirements

Layer	Technology	Justification
Frontend	Next.js, React, TypeScript	Server-side rendering helps with how fast the listing browse pages feel, which are the pages that get the most traffic. TypeScript keeps the codebase type-safe throughout.
Frontend styling	Tailwind CSS	Utility-first styling means we can iterate faster, and the token-based theming keeps things consistent with the Agile Bridge brand style guide.
Backend	NestJS, TypeScript	The module structure lines up directly with our bounded-module architecture. Decorators cut down on repetitive guard and DTO code.
Database	Azure Database for PostgreSQL (Flexible Server)	Relational data suits our domain well, and it's fully ACID-compliant. Being Azure-native also fits our hosting constraint.
ORM	TypeORM	Gives us the repository pattern for data access, plus a migration system for changing the schema over time.
Authentication	Custom OTP plus JWT	Matches the client's OTP verification requirement. JWT is stateless, so we're not tied to a managed identity provider.
File storage	Azure Blob Storage, backend-mediated with SAS tokens	Uploads go through the backend first instead of straight to Blob Storage, so storage credentials never touch the client.
Messaging	Firebase Firestore	Handles real-time chat as its own external microservice. If it goes down, the rest of the platform keeps working. The SDK also handles real-time subscriptions on its own, no backend proxying needed. It also carries the live highest bid for each auction.

Layer	Technology	Justification
Job queue	Redis (<code>redis:7-alpine</code> container) with BullMQ, through <code>@nestjs/bullmq</code> and <code>ioredis</code>	Event-driven scheduling for auction activation and closing, with no polling. Running it as a plain container inside the existing Container Apps Environment kept it cheap, and it avoids the TLS configuration branch a managed Redis service would have needed.
AI text recognition	Azure AI Vision (Image Analysis 4.0 Read API, F0 tier)	Reads title, author and ISBN from a photographed cover. Called only from the backend, so the key never reaches the client.
Client-side image processing	Custom canvas flood fill (<code>removeUniformBackground</code>)	Removes uniform backgrounds and detects book corners in the browser. We tried an ONNX background-removal model, but its size of roughly 176MB made it too slow for phones and a cold staging site.
Email	Mailtrap (sandbox on staging, live SMTP on production)	Delivers OTP codes and notification emails through one provider class, with the sandbox or live choice made by the environment.
Custom domain and TLS	Registrar DNS (CNAME and TXT records) and an Azure-managed certificate	Serves the frontend on <code>www.unitextbookmarketplace.co.za</code> over HTTPS at no extra cost.
Hosting	Azure Container Apps	Container-native, and it has a built-in revision/rollback model that we've actually had to use, which is a big part of why we chose it over App Service.
Container Registry	Azure Container Registry	Private image storage. <code>az acr build</code> lets us build remotely without needing Docker running locally.

Layer	Technology	Justification
CI (lint, test, coverage)	GitHub Actions, <code>ci.yml</code>	Runs on every push and PR to <code>main/develop</code> . Backend tests run against a real, temporary Postgres container instead of a mock, so we trust the results more.
CD (build, deploy)	GitHub Actions, <code>deploy.yml</code>	Branch-conditional, <code>main</code> deploys to production and <code>develop</code> deploys to staging. Images are tagged with the commit SHA instead of <code>:latest</code> , so Container Apps always picks up a real change.
Code quality and static analysis	SonarCloud	Runs on every push to <code>main/develop</code> and on every PR targeting either branch.
Coverage reporting	Codecov	Separate <code>backend</code> and <code>frontend</code> flags per CI job.
Testing (backend)	Jest, plus a real temporary Postgres container in CI	Gives us integration-level confidence instead of just relying on a mocked database.
Testing (frontend)	Jest, React Testing Library and Cypress	Jest and React Testing Library cover components and utilities, and Cypress runs end-to-end specs in CI.
API documentation	OpenAPI (docs/OpenAPI/openapi.yaml) and Swagger, generated from NestJS decorators	The shared <code>openapi.yaml</code> is the first source of truth, and Swagger gives interactive docs straight from the controller code.

4. API Contracts

All endpoints are prefixed with `/api`. Every authenticated endpoint needs the header `Authorization: Bearer <JWT>`. All request and response bodies use `Content-Type: application/json`. Every endpoint is also described in the shared `docs/OpenAPI/openapi.yaml`, which is version-controlled with the code.

4.1 Carried Over from Demo 1 (confirmed still in use)

4.2 Auth API Contracts

4.2.1 POST `/auth/register`

Description: Starts student registration. Checks the email domain and sends out the OTP.

Auth required: No

Request body:

```
{
  "fullName": "string (required)",
  "surname": "string (required)",
  "university": "string (required)",
  "email": "string (required, must be allowed domain)"
}
```

Success response (201 Created):

```
{
  "message": "Registration successful. Check your university email for verification code",
  "userId": "uuid"
}
```

Error responses:

- 400: Missing required field, or email domain not in allowlist
- 409: Email address is already registered

4.2.2 POST `/auth/verify-otp`

Description: Checks the 6-digit OTP sent to the student's email.

Auth required: No

Request body:

```
{
  "email": "string (required)",
  "otp": "string (required, 6 digits)"
}
```

Success response (200 OK):

```
{
  "message": "Email verified successfully.",
  "verified": true
}
```

Error responses:

- 400: OTP is incorrect or has expired
- 404: No pending OTP found for this email

4.2.3 POST /auth/login

Description: Logs in a verified student and hands back a signed JWT.

Auth required: No

Request body:

```
{
  "email": "string (required)",
  "password": "string (required, min 8 characters)"
}
```

Success response (200 OK):

```
{
  "message" : "Login sucessful."
}
```

Error responses:

- 401: Invalid email or password
- 403: Account exists but email is not verified

4.2.4 POST /auth/set-password

Description: Sets the student's password once OTP verification is done.

Auth required: No (uses `userId` from the registration step)

Request body:

```
{
  "userId": "uuid (required)",
  "password": "string (required, min 8 characters)",
  "confirmPassword": "string (required, must match password)"
}
```

Success response (200 OK):

```
{
  "message": "Password set successfully.",
  "accessToken": "string (JWT)"
}
```

4.2.5 GET /auth/me

Description: Returns the logged-in user's profile.

Auth required: Yes

4.2.6 POST /auth/logout

Description: Clears the session cookie.

Auth required: Yes

4.2.7 GET /auth/universities

Description: Lists supported universities, for the registration dropdown.

Auth required: No

4.2.8 POST /auth/refresh

Description: Gets a new access token using a refresh token, so users stay logged in without having to type their password again.

Auth required: No

4.3 Listings API Contracts

4.3.1 POST /listings

Description: Creates a new textbook listing. Gets created with `status = PENDING`.

Auth required: Yes (Student role)

Request body:

```
{
  "isbn": "string (optional)",
  "title": "string (required)",
  "author": "string (optional)",
  "edition": "integer (required)",
  "condition": "new | good | fair | poor (required)",
  "annotationLevel": "none | light | heavy (required)",
  "bookId": "uuid (required)",
  "moduleId": "uuid (required)",
  "price": "number, positive decimal (required)",
  "hasNotes": "boolean (required)",
  "photoUrls": ["string (required >= 4)"],
  "description": "string (required)"
}
```

4.3.2 GET /listings

Description: Returns all APPROVED listings, with pagination and filters.

Auth required: Yes

Query parameters: moduleId, faculty, condition, minPrice, maxPrice, edition, page, limit

4.3.3 GET /listings/mine

Description: Returns all of the logged-in student's own listings, including PENDING, APPROVED and REJECTED ones.

Auth required: Yes (Student role)

4.3.4 GET /listings/:id

Description: Returns the full detail object for one listing.

Auth required: Yes

Path parameter: id, the UUID of the listing

4.4 Admin API Contracts

4.4.1 GET /listings/admin/all

Description: Returns all listings, admin only.

Auth required: Yes (Admin role only)

4.4.2 PATCH /admin/listings/:id/approve

Description: Approves a pending listing. Sets status to APPROVED and writes an audit log entry.

Auth required: Yes (Admin role only)

4.4.3 PATCH /admin/listings/:id/reject

Description: Rejects a pending listing. Sets status to REJECTED and writes an audit log entry, a reason is included with the rejection.

Auth required: Yes (Admin role only)

Note: The audit log write was added for Demo 2

4.5 Modules API Contract

4.5.1 POST /modules

Description: Creates a module linked to the textbook being sold. Takes module details like name, code, semester, and faculty id.

4.5.2 GET /modules/faculties

Description: Returns faculty names and ids. Used when creating a module during listing creation

4.6 New for Demo 2

4.7 Listing API Contract

4.7.1 PATCH /listings/editlist

Module: Listings (UC5)

Description: Edits the details of an existing listing that the student owns. Only works on listings that are rejected or pending.

Auth required: Yes (Student role, listing owner)

Request body: Only listing details can be edited, everything is optional except the listing id.

```
{
  "id" : "string (uuid)",
  "title" : "string",
  "condition" : 'new' | 'good' | 'fair' | 'poor',
```

```
"annotation_level" : 'none' | 'light' | 'heavy',
"price" : "number",
"has_notes" : "boolean",
"description" : "string",
}
```

4.8 Module API Contract

4.8.1 GET /modules (extended filters)

Module: Smart Filters (UC6)

Description: Extends the existing module search endpoint with extra smart filter parameters.

Description: Search-as-you-type module code lookup. Returns modules matching the search query.

Auth required: Yes

Query parameters:

- **search** (string, required): partial module code or name to search
- **university** (string, optional): filter by university name (e.g. "UP")

4.9 Saved Searches API Contract

4.9.1 POST /saved-searches/

Module: Saved Search (UC6-ext)

Description: Saves a search as JSON **Request body:** all fields are optional

```
{
  "filter_json" : {
    "faculty" : "string",
    "moduleCode" : "string",
    "edition" : "string",
    "priceMin" : "string | number",
    "priceMax" : "string | number",
    "condition" : "string",
    "annotationLevel" : "string",
    "search" : "string",
    "book_title" : "string",
    "author" : "string",
    "isbn" : "string",
    "university_id" : "string",
  }
}
```

```
"faculty_id": "string"
}
}
```

4.9.2 GET /saved-searches/me

Module: Saved Search (UC6-ext)

Description: Returns the saved searches belonging to the logged-in student. Confirmed to exist as a route in `saved_search.controller.ts`.

Auth required: Yes

Request body: uses query parameters.

- `id` (string)
- `email` (string)
- `role` (string)
- `firstname` (string)
- `lastname` (string)

4.9.3 GET /saved-searches/id

Module: Saved Search (UC6-ext)

Description: Returns one saved search by id **Auth required:** Yes

Request body: uses query parameters.

- `id` (string)

4.9.4 DELETE /saved-searches/id

Module: Saved Search (UC6-ext)

Description: Deletes a saved search by id **Auth required:** Yes

Request body: uses query parameters.

- `id` (string)

4.9.5 GET /saved-searches/me/email

Module: Saved Search

Description: Returns the logged-in student's saved search email.

4.9.6 GET /saved-searches/me/role

Module: Saved Search

Description: Returns role-specific saved search settings for the logged-in user.

4.10 Wishlist API Contract

4.10.1 POST /wishlist/:listingId

Module: Wishlist (UC7)

Description: Adds a listing to the logged-in student's wishlist.

Auth required: Yes (Student role)

Request body:

```
{
  "userid" : "string (uuid)",
  "listingid" : "string(uuid)",
}
```

4.10.2 DELETE /wishlist/:listingId

Module: Wishlist (UC7)

Description: Removes a listing from the logged-in student's wishlist.

Auth required: Yes (Student role)

Request body:

```
{
  "userid" : "string (uuid)",
  "listingid" : "string(uuid)",
}
```

4.10.3 GET /wishlist/mine

Module: Wishlist (UC7)

Description: Returns the logged-in student's wishlisted listings.

Auth required: Yes (Student role)

4.11 Admin API Contract

4.11.1 GET /admin/audit-log

Module: Audit Log (UC8)

Description: Returns audit log entries for admin review.

Auth required: Yes (Admin role only)

Query parameters: all are optional

- **performedBy** (string): filter by which admin did the action
- **action** (string): approved a listing or rejected a listing
- **entityType** (string): what type the action was performed on, currently only listing
- **entityId** (string)
- **startDate** (string)
- **endDate** (string)

4.11.2 GET /admin/emails

Module: Audit Log (UC8)

Description: Returns a list of admin emails, used for filters **Auth required:** Yes (Admin role only)

4.12 Image API Contract

4.12.1 POST /images/upload

Module: Blob upload

Description: Uploads a listing image to Azure Blob Storage through the backend.

Auth required: Yes (Student role)

4.12.2 GET /images/list

Module: Blob upload

Description: Lists uploaded images for a listing.

Auth required: Yes

4.12.3 DELETE /images/delete

Module: Blob upload

Description: Deletes an uploaded listing image from Blob Storage.

Auth required: Yes (Student role, listing owner)

4.13 Conversations API Contract

4.13.1 POST /conversations

Module: Messaging

Description: Creates or fetches a conversation for a listing

Auth required: Yes

Request body:

```
{
  "listingId": "uuid"
}
```

4.13.2 POST /conversations/mine

Module: Messaging

Description: Gets all conversations for the logged-in user

Auth required: Yes

4.13.3 POST /conversations/:id/messages

Module: Messaging

Description: Gets all messages in a conversation

Auth required: Yes

Api param:

```
{
  name: 'id',
  description: 'Conversation ID',
}
```

4.13.4 GET /conversations/:id/messages

Module: Messaging

Description: Sends a message in a conversation

Auth required: Yes

Api param:

```
{
```

```
name: 'id',
description: 'Conversation ID',
}
```

Request body:

```
{
  "text" : "string"
}
```

4.14 Demo 3 API Contracts:

4.14.1 POST /reports

Module: Reports **Description:** Creates a new report **Auth required:** Yes **Request body:**

```
{
"listing_id": "uuid",
"reason": "Fraud | Misleading | Duplicate | Other"
}
```

4.14.2 GET /admin/reports

Module: Reports / Admin **Description:** Getter for reports. **Auth required:** Admin only **Query parameters:** Optional report filters.

Module: Reports / Admin **Description:** Specific getter for reports review. **Auth required:** Admin only

4.15 Admin Moderation API Contract

4.15.1 PATCH /admin/reports//dismiss

Module: Admin / Reports **Description:** Dismisses a report **Auth required:** Admin only

4.15.2 PATCH /admin//ban

Module: Admin / Moderation **Description:** Perform the ban on a report **Auth required:** Admin only **Request body:**

```
{
"reason": "string"
}
```

```
}
```

4.16 Notifications API Contract

4.16.1 GET /notifications/mine

Module: Notifications

Description: Returns the logged-in user's notifications, paginated.

Auth required: Yes

Query parameters:

- `page` (number, optional, default 1)
- `limit` (number, optional, default 10)

4.16.2 PATCH /notifications/:id/read

Module: Notifications

Description: Marks a single notification as read.

Auth required: Yes (owner only)

Path parameters:

- `id` (string, uuid): notification id

4.16.3 PATCH /notifications/read-all

Module: Notifications

Description: Marks all of the logged-in user's notifications as read.

Auth required: Yes

4.16.4 DELETE /notifications/:id/delete

Module: Notifications

Description: Deletes a notification belonging to the logged-in user.

Auth required: Yes (owner only)

Path parameters:

- `id` (string, uuid): notification id

4.17 Listing Status API Contract

4.17.1 PATCH /listings/:id/status

Module: Listings (UC5.2)

Description: Lets the seller move one of their approved listings through its sale lifecycle. RESERVED was added in Demo 3 next to the existing SOLD and WITHDRAWN values. The seller can trigger it from the messaging thread once a sale is agreed, or from My Listings, and the listing detail page shows the matching badge.

Auth required: Yes (Student role, listing owner)

Path parameter: id, the UUID of the listing

Request body:

```
{
  "status": "RESERVED | SOLD | WITHDRAWN (required)"
}
```

Error responses:

- 400 or 403: The transition is not allowed from the listing's current status, or the caller does not own the listing
- 401: Missing or invalid JWT

4.18 Demo 4 API Contracts

How to add your contract: one \subsubsection per endpoint, in this order: Module, Description, Auth required, Path or query parameters, Request body, Success response, Error responses. Add the same endpoint to docs/OpenAPI/openapi.yaml on the day it is committed. The Vision contract below is the reference layout.

Vision API Contract (Tiego Mokwena)

4.18.1 POST /vision/extract-text

Module: Vision (AI-Assisted Listing Creation, Wow Factor 1)

Description: Runs text recognition on a book cover photo that has already been uploaded through the existing image flow, then fuzzy-matches the extracted text against the Book table. The Azure AI Vision endpoint and key stay in the backend and never reach the client. The result is a suggestion only. The frontend uses it to pre-fill title, author, ISBN, edition and book id, and the seller can edit every field.

Auth required: Yes (Student role)

Request body:

```
{
```

```
"imageUrl": "string (required, URL of the uploaded cover photo)"
}
```

Success response:

```
{
  "rawText": "string",
  "matchedBook": {
    "id": "uuid",
    "title": "string",
    "author": "string",
    "isbn": "string",
    "edition": "integer",
    "confidence": "number, 0 to 1"
  } | null
}
```

Notes:

- `matchedBook` is `null` when no candidate book scores above the confidence threshold. `rawText` is still returned so the frontend can show something.
- The frontend aborts the call after about 10 seconds. On any failure or timeout it shows a non-blocking message and keeps the cropped photo, so manual entry is never blocked.
- Background removal and the draggable corner crop run entirely in the browser, so there is no second endpoint for them.

Error responses:

- 400: `imageUrl` missing or not a valid URL
- 401: Missing or invalid JWT
- 403: Caller does not have the Student role
- 5xx: The Azure AI Vision call failed. The frontend falls back to manual entry.

Auction API Contract (Gift Mohuba)

Placeholder, to be completed by Gift Mohuba. Document each auction endpoint using the layout above, at least `POST /auctions`, `GET /auctions/:id`, `POST /auctions/:id/bids` and `GET /auctions/:id/bids`. Include the auction status values, the bid validation rules and the anti-sniping behaviour in the descriptions.

Bundle Optimizer API Contract (Josh Kretschmer and Neo Bosoga)

Placeholder, to be completed by Josh Kretschmer and Neo Bosoga. Document each bundle optimizer endpoint using the layout above. Include the request shape (the wanted books), the response shape (chosen listings, total price, number of seller meetups) and the behaviour when no seller overlap exists.

Cases and Appeals API Contract (Neo Bosoga)

Placeholder, to be completed by Neo Bosoga. The appeal endpoints from Demo 3 (POST `/cases`, GET `/admin/cases` and the case decision endpoint) are not yet documented in this section. Add them using the layout above.

Other Demo 4 Contracts

Placeholder. Any other endpoint added for Demo 4 (for example profile and settings) is documented here by whoever built it.

5. Non-Functional Requirements (NFR) Testing

5.1 Testing Overview

Every quality requirement in the SRS has been quantified and verified with at least one executable, repeatable test. The table below maps each NFR to its architectural tactic and the test that validates it.

5.2 NFR Traceability Matrix

ID	Requirement	Tactic in SAS	Test Tool	Target	Status
NFR-01	GET /listings search and browse return within 2s for 95% of requests under 50 concurrent users	Composite indexing, Query optimization, Pagination, Redis caching	k6	p(95) < 2000ms	FAILED
NFR-02	Notification delivery via bell icon reflects triggering event within 15s polling interval	Polling mechanism optimization, Real-time updates	k6	< 15s	UNTESTED
NFR-03	Stateless JWT-based API layer supports horizontal scaling without code changes	No session state on server, JWT tokens	Architecture Review	Pass	PASS
NFR-04	Production runs with min-replicas: 1, max-replicas: 2 on Azure Container Apps	Container orchestration, Auto-scaling	Azure CLI	Configured	PASS
NFR-05	Staging scales to zero when idle (min-replicas: 0) with cold-start delay 20s	Zero-scaling configuration	Azure CLI	< 30s	PASS
NFR-06	All protected endpoints enforce JWT-based authentication	JWT middleware, Token validation	k6	401/403 for invalid/no token	UNTESTED

ID	Requirement	Tactic in SAS	Test Tool /	Target	Status
NFR-07	Admin-only end-points return 403 Forbidden for Student-role JWT	Role-based access control (RBAC)	k6	403 Forbidden	UNTESTED
NFR-08	Passwords hashed with bcrypt at cost factor 12	bcrypt encryption, Salt generation	Jest	Cost factor 12	PASS
NFR-09	Login, registration, password-change end-points rate-limited to 5 req/min per client	Rate limiting middle-ware	k6	429 after 5 requests	UNTESTED
NFR-10	Access tokens expire after 15 minutes, re-fresh tokens after 7 days	JWT expiry configu-ration	Jest	15min / 7 days	PASS
NFR-11	Banned user (is_banned=true) rejected with 403 on every login attempt	User status check middleware	k6	403 Forbidden	PASS
NFR-12	Core features remain available if external messaging microser-vice is unavailable	Circuit breaker, Fall-back mechanism	Jest + Fault Injection	No 5xx er-rors	UNTESTED
NFR-13	Production main-tains at least 1 warm replica at all times	Minimum replica con-figuration	Azure CLI	1 replica	PASS
NFR-14	Failed deployment rollback-able to known-good revision in under 5 minutes	Azure revision traffic shifting	Azure CLI	< 5 min-utes	UNTESTED
NFR-15	Backend test cover-age measured on ev-ery push/PR via CI pipeline	CI/CD pipeline, Codecov integration	Codecov	80% cover-age	FAILED

ID	Requirement	Tactic in SAS	Test Tool /	Target	Status
NFR-16	Every new endpoint has corresponding entry in openapi.yaml same day	API documentation, OpenAPI spec	Manual Review	100% documented	PASS
NFR-17	UI meets Google Lighthouse Accessibility score 85/100 on every audited page	Semantic markup, ARIA labels	axe-core / Lighthouse	85/100	PASS
NFR-18	New pages from Demo 3 onward score 90/100 on Lighthouse	Accessibility-first design	axe-core / Lighthouse	90/100	UNTESTED
NFR-19	Every moderation, ban, appeal-decision action writes exactly one immutable audit_log entry	Audit log middleware	Jest	1 row per action	PASS
NFR-20	Audit log entries record action, admin UUID, and timestamp	Structured logging	Jest	All fields present	PASS

5.3 Test Results Summary

Category	Total	PASS	FAIL	UNTESTED
Performance	2	0	1	1
Scalability	3	3	0	0
Security	6	3	0	3
Reliability/Availability	2	1	0	1
Maintainability	3	1	1	1
Accessibility	2	1	0	1
Auditability	2	2	0	0
TOTAL	20	11	2	7

5.4 Load Test Results

Test Configuration:

- **Duration:** 40.7 seconds
- **Virtual Users:** 5 concurrent (ramping from 1 to 5)
- **Total Iterations:** 25
- **Success Rate:** 80% (120 out of 150 checks passed)

Endpoint Performance:

Endpoint	Avg	p(95)	Status
/listings	3.62s	19.99s	FAILED
/auth (JWT)	60.47ms	265.51ms	PASS
/auth/universities	43.95ms	199.89ms	PASS

5.5 Accessibility Test Results

Pages Tested:

Page	Lighthouse Score	axe-core Violations
Landing Page	94/100	0
Create Listing	94/100	0
In-App Messaging	95/100	0
Wishlist	96/100	0
Listings Browse	89/100	0
Audit Log	85/100	0

5.6 Testing Tools Used

Tool	Purpose	NFRs Covered
k6	Load testing for performance metrics under concurrent users	QR-01, QR-02, QR-06, QR-07, QR-09, QR-11
Jest	Unit & integration tests for security, auditability, fault injection	QR-08, QR-10, QR-12, QR-19, QR-20
axe-core / Lighthouse	Automated accessibility scanning for WCAG 2.1 AA compliance	QR-17, QR-18
Codecov	Test coverage tracking on CI pipeline	QR-15

Tool	Purpose	NFRs Covered
Azure CLI	Infrastructure validation for scaling & availability	QR-03, QR-04, QR-05, QR-13, QR-14
Manual Review	OpenAPI specification verification	QR-16

5.7 Known Issues and Action Plan

5.7.1 Critical Issues (Need Immediate Fix)

- **NFR-01:** Listings endpoint p95 latency at 19.99s (target <2000ms)
 - **Root Cause:** Missing database indexes on listings table, potential N+1 query pattern
 - **Action:** Add composite indexes, implement eager loading, add Redis caching
- **NFR-15:** Test coverage unknown (target 80%)
 - **Root Cause:** Coverage report not generated/pushed to Codecov
 - **Action:** Run `npm run test:cov` and push to Codecov

5.7.2 Untested NFRs (Need Testing)

- **NFR-02:** Notification polling 15s
- **NFR-06:** JWT authentication testing
- **NFR-07:** Admin RBAC testing
- **NFR-09:** Rate limiting testing
- **NFR-12:** Circuit breaker testing
- **NFR-14:** Rollback testing
- **NFR-18:** New page accessibility

6. Deployment

6.1 Overview

The system is deployed as two separate services, a NestJS backend and a Next.js frontend, both packaged into Docker images and run on Azure Container Apps. A managed Azure Database for PostgreSQL Flexible Server handles the data, and Azure Blob Storage handles listing image uploads through the backend, using SAS-token authentication. A Redis container runs inside the same Container Apps Environment and backs the auction job queue, and Azure AI Vision provides text recognition for AI-assisted listing creation.

There are two environments, production (served off `main`) and staging (served off `develop`), which is what covers the environment parity requirement. Both share one Container Apps Environment to keep costs down, but they're fully separated at the application and data level.

	Production	Staging
Trigger branch	<code>main</code>	<code>develop</code>
Backend app	<code>nexusdev-backend</code>	<code>nexusdev-backend-staging</code>
Frontend app	<code>nexusdev-frontend</code>	<code>nexusdev-frontend-staging</code>
Database	<code>nexustextbook</code>	<code>nexustextbook_staging</code>
Scaling	<code>min-replicas: 1,</code> always warm	<code>min-replicas: 0,</code> scales to zero when idle
Job queue store	<code>nexusdev-redis</code> (shared, internal ingress only)	<code>nexusdev-redis</code> (shared, internal ingress only)
AI text recognition	<code>nexusdev-vision</code> (shared, F0 tier)	<code>nexusdev-vision</code> (shared, F0 tier)

Redis and Azure AI Vision are shared by both environments, the same way the Blob Storage account is. Each backend app gets the Redis host and port as environment variables, and the Vision endpoint and key as Container Apps secrets, so the values are set once per app and survive every deploy, because the pipeline only updates the image.

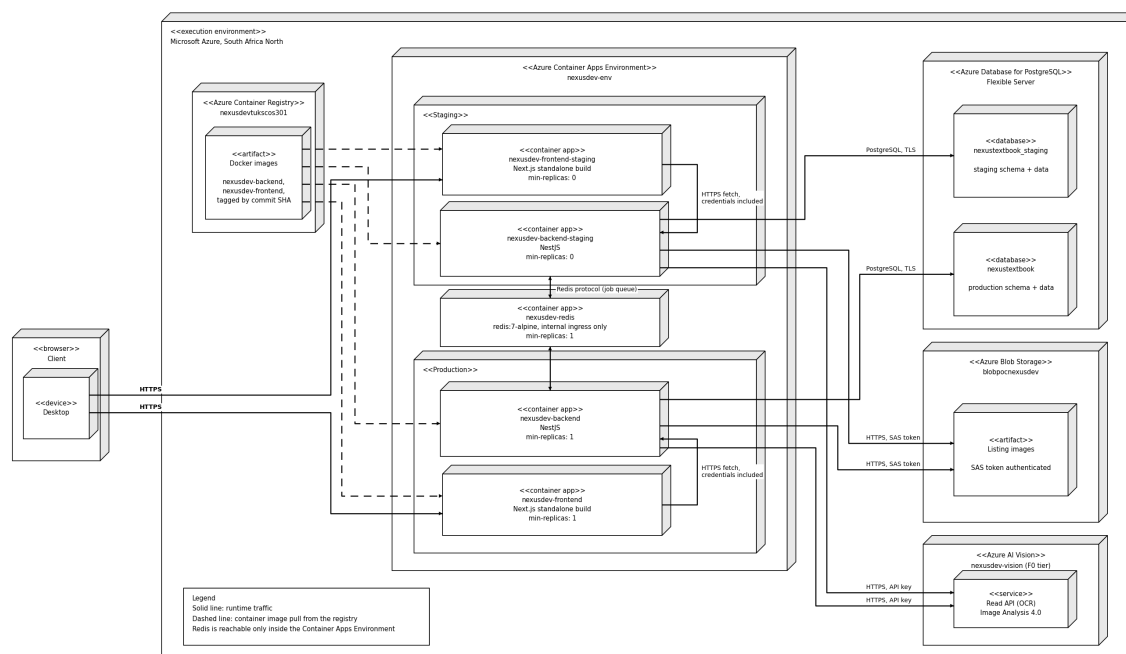
We went with one shared Container Apps Environment and separate databases on purpose, instead of two fully separate environments. A second full environment felt like unnecessary cost for a student capstone project, and the part that actually matters for correctness, keeping the data isolated, is already handled by giving each environment its own Postgres

database on the same server. Staging runs at `min-replicas: 0`, which means the first request after it's been idle a while takes a bit longer (a cold start), but it costs basically nothing while no one's using it. Production stays warm at `min-replicas: 1` since it needs to respond right away for real users at any time.

6.2 Live URLs

- **Production frontend (custom domain):** <https://www.unitextbookmarketplace.co.za>
- **Production frontend (default hostname):** <https://nexusdev-frontend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io>
- **Production backend:** <https://nexusdev-backend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api>
- **Staging frontend:** <https://nexusdev-frontend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io>
- **Staging backend:** <https://nexusdev-backend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api>

6.3 Deployment Diagram

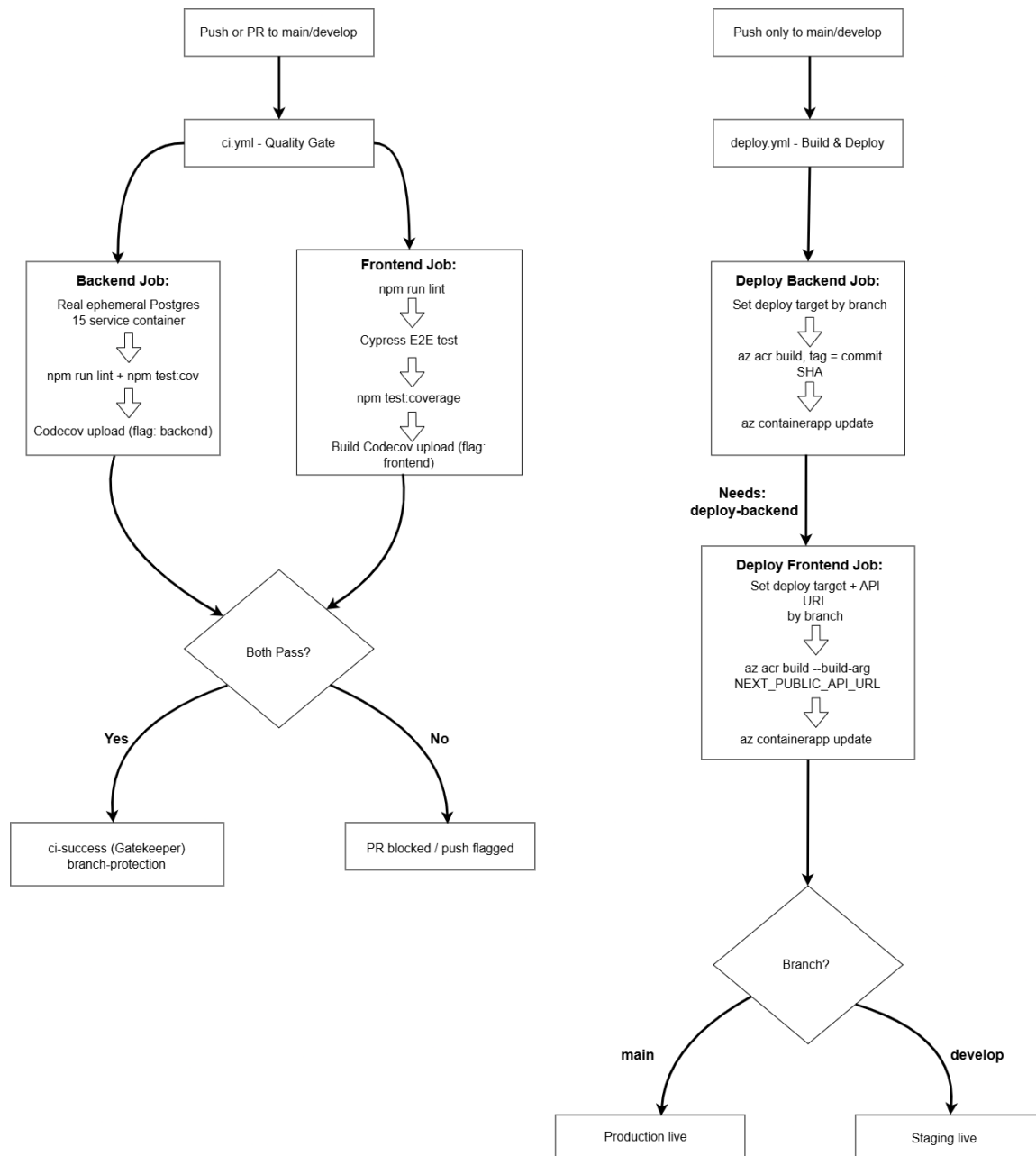


This one diagram covers both environments, since they run on the exact same topology and only differ in which specific app and database instance they point at (labelled per node above). Both backends also talk to the shared `nexusdev-redis` container over the

Container Apps Environment's internal network, and to Azure AI Vision over HTTPS with an API key. Region is South Africa North, resource group is NexusDev.

6.4 CI/CD Pipeline Diagram

There are two separate GitHub Actions workflows here, each doing a different job.



A few notes on how this actually works:

- `ci.yml` runs on both push and pull request to `main/develop`, this is the real quality gate. It runs the backend and frontend Jest jobs, the Cypress end-to-end specs, and SonarCloud. `deploy.yml` only runs on push, since an open PR shouldn't trigger an actual deployment.

- Branch protection is set up on both `main` and `develop`. Every merge has to come through a PR from a feature branch, needs at least one reviewer to approve it, and the `ci-success` check has to pass. A failed CI run blocks the merge outright.
- Inside `deploy.yml`, `deploy-frontend` only runs after `deploy-backend` succeeds, since the frontend build needs the backend's live URL as a build-time argument.
- Every image is tagged with the full commit SHA, never `:latest`. This was actually a real bug we ran into early on, Container Apps sometimes wouldn't notice that a `:latest` image had changed, so we switched to SHA tags to make every deploy unambiguous.

6.5 Deployment Requirements Status

Requirement	Status	Notes
Live, accessible system	Done	Both environments reachable over public HTTPS (§6.2)
Environment parity	Done	Production and staging each have their own Container Apps, their own database, and their own CI/CD trigger
Infrastructure as Code / Containerization	Done	Docker multi-stage builds, deployed through Azure Container Apps
Secrets management	Done	All credentials live as Container Apps secrets or GitHub Actions secrets, never committed to the repo. This now includes the Vision key and endpoint.
Job queue infrastructure	Done	Redis runs as an internal-only container in the Container Apps Environment, and both backends are configured with its host and port
Custom domain with HTTPS	Done	Frontend served on www.unitextbookmarketplace.co.za with a managed certificate (§6.6)
Rollback strategy	Done	See §6.7

6.6 Custom Domain and DNS

The production frontend is also served from <https://www.unitextbookmarketplace.co.za>. DNS is managed in the registrar's control panel (Absolute Hosting). Two records were added for the `www` host: a TXT record (`asuid.www`) carrying the Container Apps domain

verification ID, and a CNAME record (**www**) pointing at the frontend’s default Container Apps hostname. Once both resolved, the hostname was added to the frontend Container App and a free Azure-managed TLS certificate was bound to it using CNAME validation.

Standard DNS does not allow a CNAME on the bare domain, so `www.unitextbookmarketplace.co.za` is the canonical address and is the one used in the README and the demo. Two knock-on changes were needed. The backend’s CORS origins in `main.ts` now include the custom domain, and the Blob Storage CORS rules do too, because the photo crop editor reads image pixels through a canvas and would fail on a blocked origin. The backend keeps its default Container Apps hostname, so `NEXT_PUBLIC_API_URL`, which is baked in at frontend build time, did not need to be updated.

6.7 Rollback Strategy

Azure Container Apps keeps old revisions around. If a deployment goes wrong, we can just shift traffic back to the last known good revision without rebuilding anything:

```
az containerapp revision list --name nexusdev-backend --resource-group NexusDev
  --output table
az containerapp ingress traffic set --name nexusdev-backend --resource-group
  NexusDev --revision-weight <previous-revision-name>=100
```

This isn’t just theoretical, we actually used this exact process during initial production deployment to fix a stale image that was causing CORS issues.

6.8 Issues Encountered and the Decisions That Came Out of Them

Issue	Root Cause	Decision Made
Stale <code>:latest</code> tag not picked up as changed by Container Apps	The tag string itself never changes, so the platform assumed nothing had	Every image now gets tagged with the commit SHA instead
Cross-origin cookie not keeping people logged in after login	<code>sameSite: 'lax'</code> blocks cross-origin fetch, and frontend and backend end up on different hostnames once deployed	Cookie settings now change per environment, <code>sameSite: 'none'</code> and <code>secure: true</code> when deployed, <code>'lax'</code> locally

Issue	Root Cause	Decision Made
Fresh database had no seed data, so registration kept failing	The seed scripts had never actually been run against a genuinely empty database before	Confirmed the seed scripts work properly against both a fresh staging database and a patched production database
Two migrations failed the same way on staging and, it turned out, on production too	Migration class names didn't match their required timestamp suffix, and a leftover name property on the class kept overriding the fix even after the class name was already corrected	Fixed both migration files properly, and manually applied the corrected SQL straight to production since it had the exact same problem
The frontend image accidentally got deployed into the backend's Container App on staging	A copy-paste mistake in deploy.yml's branch logic set the wrong app name in one spot	Fixed the value, then double-checked with az containerapp show that each app was finally running the right image
Staging was costing money even while nobody was using it	min-replicas: 1 keeps a Container App running the whole time no matter what	Set min-replicas: 0 on both staging apps, accepting a short cold start on the first request after it's been idle
Auctions needed to close at an exact time, and anti-sniping keeps moving that time	A polling cron job wastes work, and cancelling and re-adding a delayed job races when two extensions land close together	Event-driven BullMQ jobs, with a close job that re-checks the end time and reschedules itself, so it can never fire early or twice

Issue	Root Cause	Decision Made
Bid transaction failed with "FOR UPDATE cannot be applied to the nullable side of an outer join"	The locking query joined the seller and highest-bidder relations, and Postgres refuses to lock a nullable joined side	Removed the joins from the locked query and exposed the foreign key ids as plain columns on the entities
Auction feature could not run on Azure because there was no Redis there	Redis existed only as a local Docker container	Provisioned Redis as an internal-only container app in the existing environment. We chose it over a managed Redis service because that needs TLS on a different port, a code branch between dev and production, and a monthly cost
Background removal model stalled and was far too heavy for phones	The ONNX model was roughly 176MB and timed out after 30 seconds	Dropped it for a dependency-free flood fill that runs in the browser and falls back to manual corners on busy backgrounds
Crop confirm could fail with a tainted canvas	Photos load from Blob Storage and are read through a canvas, which needs CORS headers	Added every deployed frontend origin, including the custom domain, to the storage account's CORS rules

Uni Textbook Marketplace

In collaboration with:



2026 NexusDev - University of Pretoria - COS 301 Capstone Project
This document is version-controlled in the GitHub repository under
[/docs/Demo_4/Software Architecture Specifications.pdf](#)